

Week 3 Lab: Control Flow & Functions

Student ID: 28816

Program: Introduction to Big Data

Date: September 18, 20261

1. The 3 Broken Records

After running the resilient pipeline (Exercise 5.1), the following **3 records** were identified as broken because their score column (index 7) could not be converted to an integer:

	Student ID	Name	Problem
1	AUCA012	Emmanuel Nsengiyumva	Score value was “N/A”
2	AUCA025	Samuel Rukundo	Score value was empty
3	AUCA038	Gilbert Uwituze	Score value was “absent”

2. How My Code Survived Them

The pipeline used a try/except ValueError block around the score conversion:

```
try:
    score = int(parts[7])
    total += score
    count_valid += 1
except ValueError:
    bad += 1
    bad_ids.append(parts[0])
```

Why this works:

- When `int("abc")` or `int("")` is called, Python raises a `ValueError` instead of crashing.
- The try block attempts the conversion; if it succeeds, the score is added to the running total and the valid counter is incremented.
- If it fails, the except block catches the error, increments the bad counter, and records the student ID in `bad_ids`, **without stopping the loop**.
- This means the program processes all 40 rows and reports both the valid statistics (average, passed/failed, top student) and the list of broken records.

Results:

- **Valid records:** 37
- **Broken records:** 3
- **Bad IDs:** ['AUCA012', 'AUCA025', 'AUCA038']
- **Average of valid scores:** 75.3

3. What I Learned

This lab showed me that real-world data is rarely clean. Instead of letting one bad row crash the whole program, the try/except pattern lets the pipeline **skip the broken record, keep going, and report the problem afterward**. This is far better than silently dropping data, the bad_ids list makes the issue visible so it can be investigated later.

I also learned that the **accumulator pattern** (initialize before the loop → update inside → report after) is the backbone of almost every data-processing task, from counting passes to averaging scores by district.

4. Reflection (Part 6)

At what percentage would I stop trusting the dataset?

If more than **10%** of records were broken, I would stop trusting the dataset. At that point, skipping them would bias the results too much. Instead of skipping, I would try to recover the values (check for typos, decimals, or "N/A" markers), flag them for manual review, and report the missing rate alongside any analysis.

One thing that surprised me:

The try/except ValueError pattern was surprising, it lets the program keep running even when data is broken, instead of crashing. It's a simple but powerful way to build resilient pipelines.

End of report